



EC-220 Computer System Architecture

Dr. Ahsan Shahzad

Department of Computer and Software Engineering (DC&SE)

College of Electrical and Mechanical Engineering (CEME)

National University of Science and Technology (NUST)

Week # 9_OLE

Floating Point

- **Scientific Notation**

- has a single digit to the left of the decimal point

- **Normalized**

- Has no leading 0's
- 1.0×10^{-9} (o) 0.1×10^{-8} (x) 10.0×10^{-10} (x)

- Binary Nos. in scientific notation

$$1.0_{\text{two}} \times 2^{-1}$$

- In computer arithmetic, such nos. are called **floating point** as the binary point is not fixed, as it is for integers

$$3.14159265..._{\text{ten}} \text{ (pi)}$$

$$2.71828..._{\text{ten}} \text{ (e)}$$

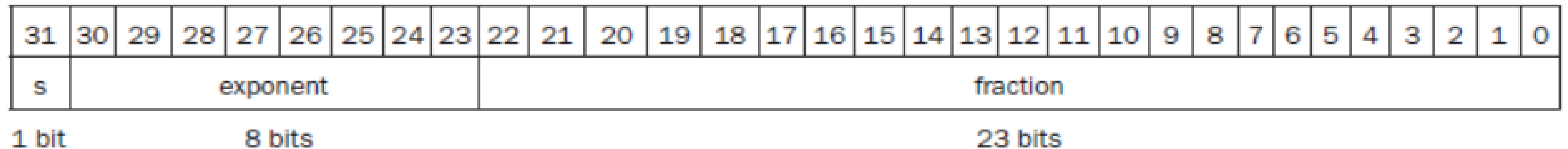
$$0.000000001_{\text{ten}} \text{ or } 1.0_{\text{ten}} \times 10^{-9} \text{ (seco)}$$

$$3,155,760,000_{\text{ten}} \text{ or } 3.15576_{\text{ten}} \times 10^9$$

Floating Point Representation

- Sign and Magnitude format
- Compromise between size of exponent and fraction
- Tradeoff between Range and Precision
- General Form:

$$(-1)^s \times F \times 2^E$$

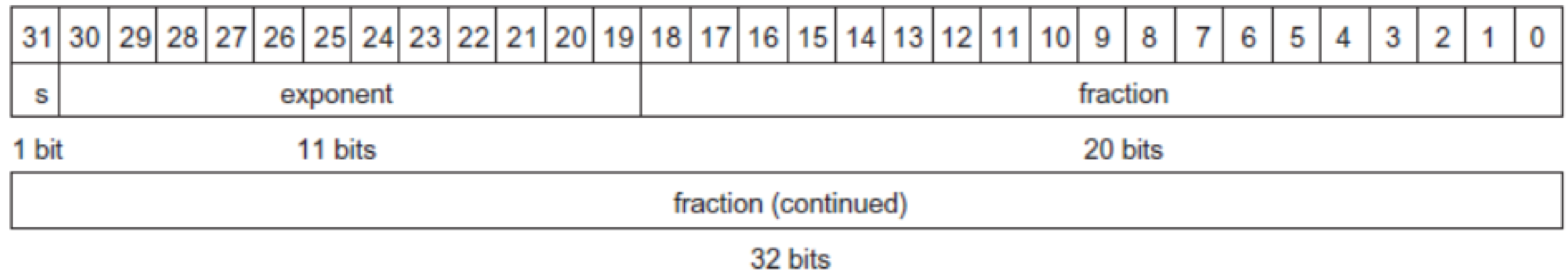


- A more better representation is discussed later in this presentation

Floating Point Representation

$$(-1)^s \times F \times 2^E$$

- **Single Precision**
 - FP value represented in a single 32-bit word
- **Double Precision**
 - FP value represented in two 32-bit words
- **Underflow or Overflow**
 - (-ve or +ve) Exponent is too large to fit in exponent field



IEEE 754 floating-point Standard

$$(-1)^s \times (1 + \text{Fraction}) \times 2^E$$

- Reserves symbols to represent unusual events
 - Infinity (overflow-> /0) , NAN (0/0)

Single precision		Double precision		Object represented
Exponent	Fraction	Exponent	Fraction	
0	0	0	0	0
0	Nonzero	0	Nonzero	± denormalized number
1–254	Anything	1–2046	Anything	± floating-point number
255	0	2047	0	± infinity
255	Nonzero	2047	Nonzero	NaN (Not a Number)

FIGURE 3.13 IEEE 754 encoding of floating-point numbers. A separate sign bit determines the

$$(-1)^s \times (1 + (s1 \times 2^{-1}) + (s2 \times 2^{-2}) + (s3 \times 2^{-3}) + (s4 \times 2^{-4}) + ...) \times 2^E$$

IEEE 754 floating-point Standard

- Utilizes Biased Notation
 - For easy integer comparisons, especially in sorting
 - Sign bit as MSB, allows quick test of less than, greater or equal to 0 comparisons
 - Next exponent comparison for FP nos. sorting
 - Negative exponents pose a challenge to simplified sorting
 - E.g. if we use 2's complement to represent –ve exponent, it will look big number in integer instr.

$$1.0 \times 2^{+1}$$

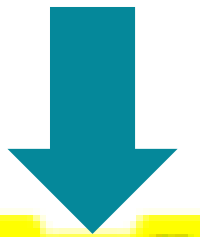
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	.	.	.

$$1.0 \times 2^{-1}$$

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	.	.	.

IEEE 754 floating-point Standard

$$(-1)^S \times (1 + \text{Fraction}) \times 2^E$$



$$(-1)^S \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$$

- **Biased Notation**

- The most negative exponent as 00..000s
- The most +ve exponent as 11..1111s
- **Biased exponent:**
 - Adding bias value 127 (Single-Precision) or 1023 (Double-Precision) to the exponent.
 - E.g. exponent -1 in biased exp. Notation will be $-1 + 127 \rightarrow 126 = 0111\ 1110$
 - And +1 will be $+1 + 127 \rightarrow 128 = 1000\ 000$

The range of single precision numbers is then from as small as

$$\pm 1.0000000000000000000000000000_{\text{two}} \times 2^{-126}$$

to as large as

$$\pm 1.1111111111111111111111111111_{\text{two}} \times 2^{+127}.$$

Floating Point Addition

From Book Page 203-207

Rounding-Off in Binary Floating Point Numbers

There are five rounding methods in IEEE 754 standard

They are easy to understand with decimal numbers example.

Example of rounding to integers using the IEEE 754 rules

Mode / Example Value	+11.5	+12.5	-11.5	-12.5
to nearest, ties to even	+12.0	+12.0	-12.0	-12.0
to nearest, ties away from zero	+12.0	+13.0	-12.0	-13.0
toward 0	+11.0	+12.0	-11.0	-12.0
toward $+\infty$	+12.0	+13.0	-11.0	-12.0
toward $-\infty$	+11.0	+12.0	-12.0	-13.0

Rounding-Off in Binary Floating Point Numbers

Default mode for binary floating point numbers

- **round to nearest; ties to even**

General Rule:

When rounding binary fractions to n-th place, check the n+1-th place number.

- If it's 0, always rounded down
- If it's 1 and there are other subsequent 1's, then rounded up
- If it's 1 but all other subsequent bits are 0's, then ties to even / tie breaking rule

Example: let's round some numbers to **2** places after the radix point:

- **0.11001** — rounds down to **0.11**, because the digit at the **3**-rd place is **0**
- **0.11101** — rounds up to **1.00**, because the digit at the **3**-rd place is **1** and there are following digits of **1** (**5**-th place)
- **0.11100** — apply the 'ties to even' tie breaker rule and round up because the digit at **3**-rd place is **1** and the following digits are all **0**'s.

Appendix-I

- <https://indepth.dev/how-to-round-binary-numbers/>

Ties to even

Sometimes the distance between the original number and each rounding option is equal, so the rule '*round to the nearest*' can't be applied. Suppose we're rounding the number **0.11011** to **4** places after the radix point. We know that we need to find two rounding options and the middle for comparison. So, to find first the smaller number we simply truncate the remaining digits after the **4**-th place which gives us **0.1101**. Now we can find the middle by appending **1** to the end of the lower number and we get **0.11011**. And the last, let's find the bigger number by adding **1** ULP to the smaller number — **0.1101+0.0001 = 0.1110**.

So here we have:

- original number — **0.11011**
- number to round down to — **0.1101**
- number to round up to — **0.1110**
- middle number between rounding options — **0.11011**.

By going through bit by bit comparison, we can see that the original number and the middle are equal, which shows that the original number falls right in between two rounding options. This is exactly the situation where we need to apply the second part of the rule '*ties to even*'. Binary number is even if it ends with **0**, so given our rounding options it's the bigger number that is even and the rule instructs to round up to the **0.1110**.